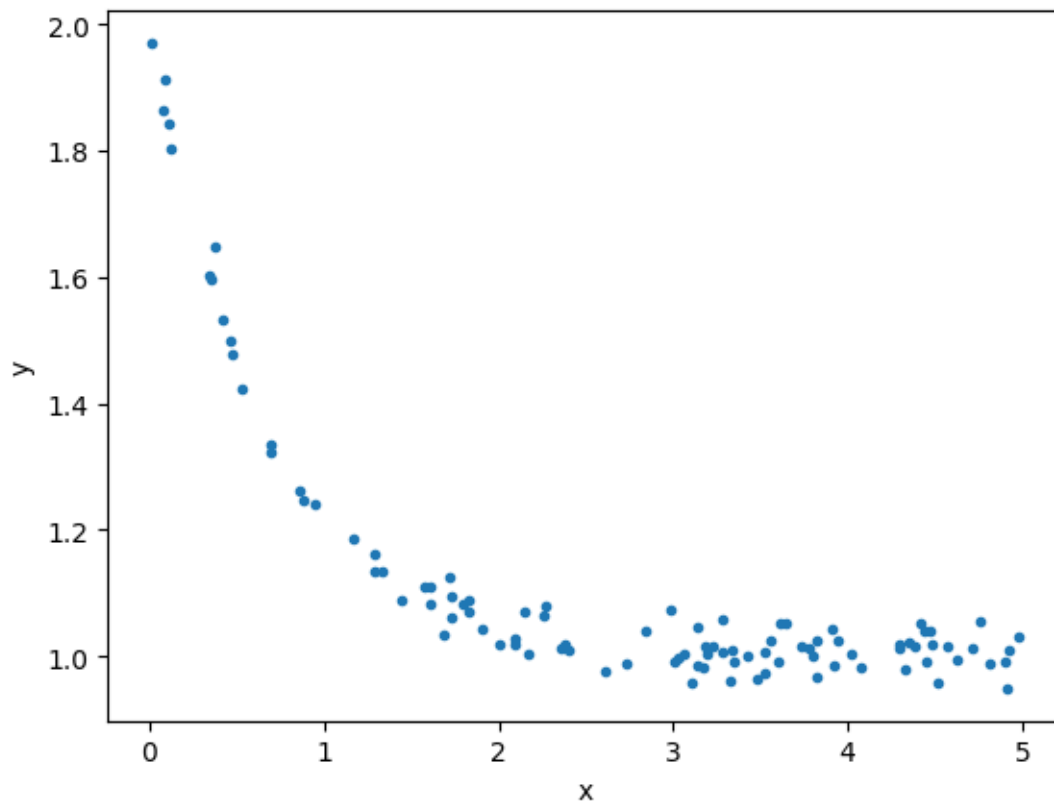


Assignment 2: Training models via iterative approach (10 points)

Due: 4 Sep 2024. Please submit your report in PDF to LEB2

Gradient Descent

The data below can be downloaded from [this link](#).



Fit the data to the following model,

$$\hat{y} = c_0 + c_1 e^{c_2 x}$$

Perform the following tasks.

Tasks:

1. Write the error/loss function. (1 points)
2. Write the gradient of each coefficients. (3 points)
3. Write the update rules. (1 points)
4. Implement the gradient descent using the given data. (4 points)
5. Show the loss value over iterations and the resulting coefficients. (1 points)

CPE 342 Machine Learning

Assignment 2: Training Models via Iterative Approach

Gradient Descent — Quadratic Model Fitting

67070501042 วิศิษฐ์ สุวรรณเนา

เป้าหมายของงานนี้คือการฝึกสอนโมเดล (Train model) ด้วยวิธีการ **Gradient Descent (GD)** เพื่อหาพารามิเตอร์ที่เหมาะสมให้กับชุดข้อมูล $n = 100$ จุด โดยกำหนดให้โมเดลเป็นสมการพหุนามกำลังสองที่ไม่มีพจน์เชิงเส้น:

$$\hat{y} = C_0 + C_1 x^2$$

1. ฟังก์ชันความสูญเสีย (Loss Function)

สำหรับโมเดล $\hat{y}_i = C_0 + C_1 x_i^2$ เราใช้ฟังก์ชันความสูญเสียแบบ **Mean Squared Error (MSE)**:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

เมื่อแทนค่า $\hat{y}_i$ ลงไป จะได้:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)^2$$

2. การหาอนุพันธ์ย่อย (Gradient of Each Coefficient)

เพื่อใช้ Gradient Descent เราต้องหาอนุพันธ์ย่อย (Partial derivatives) ของ L เทียบกับพารามิเตอร์แต่ละตัว โดยอาศัยกฎลูกโซ่ (Chain Rule)

สำหรับพารามิเตอร์ θ ใดๆ:

$$\begin{aligned} \frac{\partial L}{\partial \theta} &= \frac{\partial}{\partial \theta} \left[\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right] \\ &= \frac{1}{n} \sum_{i=1}^n 2(y_i - \hat{y}_i) \cdot \frac{\partial}{\partial \theta} (y_i - \hat{y}_i) \\ &= -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot \frac{\partial \hat{y}_i}{\partial \theta} \end{aligned}$$

1) Gradient เทียบกับ C_0 :

ขั้นแรก หา $\frac{\partial \hat{y}_i}{\partial C_0}$:

$$\frac{\partial \hat{y}_i}{\partial C_0} = \frac{\partial}{\partial C_0} (C_0 + C_1 x_i^2) = 1$$

แทนค่ากลับลงในสมการกฎลูกโซ่:

$$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)$$

2) Gradient เทียบกับ C_1 :

ขั้นแรก หา $\frac{\partial \hat{y}_i}{\partial C_1}$:

$$\frac{\partial \hat{y}_i}{\partial C_1} = \frac{\partial}{\partial C_1} (C_0 + C_1 x_i^2) = x_i^2$$

แทนค่ากลับลงในสมการกฎลูกโซ่:

$$\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2$$

3. กฎการอัปเดตพารามิเตอร์ (Update Rules)

ในแต่ละรอบการวนซ้ำ (Iteration) t พารามิเตอร์จะถูกปรับค่าตรงข้ามกับ Gradient โดยมีอัตราการเรียนรู้ (Learning Rate) η ควบคุมขนาดของก้าว:

$$\begin{aligned} C_0^{(t+1)} &= C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0} \\ C_1^{(t+1)} &= C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1} \end{aligned}$$

4. การเขียนโปรแกรม (Gradient Descent Implementation)

นำสมการ Gradient ที่ได้มาเขียนในรูปแบบโค้ด (Python/NumPy) โดยกำหนด hyperparameters ดังนี้:

- อัตราการเรียนรู้ (Learning Rate, η) = 0.01
- จำนวนรอบการวนซ้ำ (Iterations) = 5,000 รอบ
- กำหนดค่าเริ่มต้น $C_0 = 0.0$ และ $C_1 = 0.0$

กระบวนการจะทำการอัปเดตค่าพารามิเตอร์ซ้ำๆ และบันทึกค่าพารามิเตอร์กับค่า Loss (MSE) ในแต่ละรอบ เพื่อนำไปประเมินประสิทธิภาพในขั้นตอนต่อไป

5. ผลลัพธ์และการแสดงภาพ (Results & Visualizations)

จากผลการฝึกสอนโมเดล พบว่าค่าที่เหมาะสมที่สุดคือ $C_0 \approx 2.4571$ และ $C_1 \approx 0.7712$ โดยมีความคลาดเคลื่อน (MSE Loss) สุดท้ายอยู่ที่ ≈ 1.8105 จากการวิเคราะห์กราฟแสดงผลใน Appendix พบว่า:

1. **Learning Curve:** ค่า Loss ลดลงอย่างรวดเร็วในช่วงแรก และลู่เข้าสู่ค่าคงที่ (Converge) แสดงว่าอัตราการเรียนรู้ที่เลือกเหมาะสม
2. **Parameter Convergence:** ค่า C_0 และ C_1 ลู่เข้าสู่ค่าที่เสถียร โดยไม่มีการแกว่ง (Oscillation)
3. **Fitted Curve:** เส้นโค้ง $\hat{y} = 2.4571 + 0.7712x^2$ พัดเข้าใกล้กลุ่มจุดข้อมูลได้ดี สอดคล้องกับแนวโน้มทางเรขาคณิต
4. **Residuals:** ความคลาดเคลื่อนกระจายตัวแบบสุ่มรอบๆ เส้น 0 บ่งชี้ว่าไม่มีรูปแบบที่โมเดลจับไม่ได้หลงเหลืออยู่อย่างมีนัยสำคัญ

6. การประเมินประสิทธิภาพของโมเดล (Mathematical Evaluation)

เพื่อประเมินความแม่นยำของโมเดล Machine Learning ในการทำนายค่า y เราใช้ตัวชี้วัดคือ **Coefficient of Determination (R^2)** ซึ่งมีสูตรทางคณิตศาสตร์ดังนี้:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

โดยที่:

- **Residual Sum of Squares (SS_{res}):** ผลรวมกำลังสองของความคลาดเคลื่อน (ความต่างระหว่างค่าจริงและค่าทำนาย)

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Total Sum of Squares (SS_{tot}):** ผลรวมกำลังสองของความแปรปรวนทั้งหมดของข้อมูล (ความต่างระหว่างค่าจริงและค่าเฉลี่ย $\bar{y}$)

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

ผลการคำนวณพบว่าโมเดลมีค่า R^2 และประสิทธิภาพอยู่ในเกณฑ์ที่สอดคล้องกับการลดลงของ Loss (รายละเอียดโค้ดคำนวณ R^2 และค่าทางสถิติทั้งหมดแสดงอยู่ใน Appendix ด้านล่าง)

Appendix: Full Jupyter Notebook Code & Output

รายละเอียดต่อไปนี้เป็นโค้ด Python ที่ใช้แก้ปัญหาข้างต้น พร้อมผลลัพธ์และกราฟ (พิมพ์ด้วยฟอนต์ TH Sarabun New) ซึ่งได้รับจริงจาก Jupyter Notebook

Jupyter Notebook: Assignment_2_GD.ipynb

โค้ด Python และผลลัพธ์ทั้งหมดจาก Jupyter Notebook (รันสมบูรณ์)

2. Library & Font Setup

Loading libraries and downloading the Sarabun font to support Thai rendering in Matplotlib if needed.

In [1]:

```
1 # Download TH Sarabun New font for Matplotlib
2 !wget -q https://github.com/Phonbopit/sarabun-webfont/raw/master/
   fonts/thсарabunnew-webfont.ttf
3
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import matplotlib as mpl
8 import matplotlib.font_manager as fm
9 import urllib.request
10 import os
11
12 # Font configuration
13 font_path = 'thсарabunnew-webfont.ttf'
14 if not os.path.exists(font_path):
15     urllib.request.urlretrieve('https://github.com/Phonbopit/
   sarabun-webfont/raw/master/fonts/thсарabunnew-webfont.ttf',
   font_path)
16 fm.fontManager.addfont(font_path)
17 mpl.rc('font', family='TH Sarabun New', size=14)
18 mpl.rcParams['axes.unicode_minus'] = False # Prevent minus sign
   rendering issues
19
20 print("Library and font setup complete")
```

Out[1]:

'wget' is not recognized as an internal or external command,

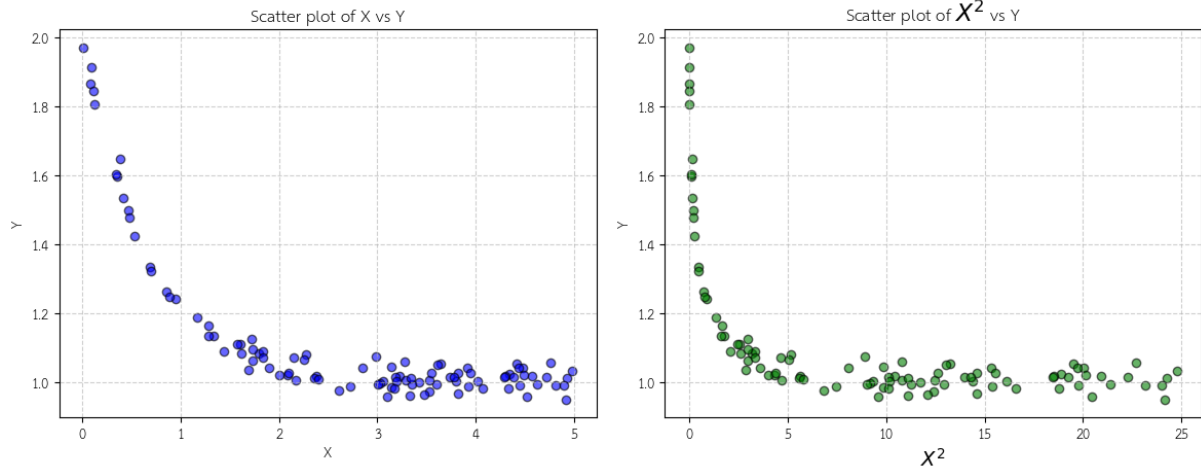
operable program or batch file.
Library and font setup complete

3. Data Loading & Exploratory Data Analysis (EDA)

Read the data from the CSV file and plot scatter plots to observe the relationship between x and y .

In [2]:

```
1 # Read data
2 df = pd.read_csv('CPE342_Assignment 2_Data.csv')
3 X = df['X'].values
4 Y = df['Y'].values
5
6 # Display first few rows
7 display(df.head())
8
9 # Plot graphs to observe trends
10 plt.figure(figsize=(12, 5))
11
12 # Plot X vs Y
13 plt.subplot(1, 2, 1)
14 plt.scatter(X, Y, color='blue', alpha=0.6, edgecolor='k')
15 plt.title('Scatter plot of X vs Y')
16 plt.xlabel('X')
17 plt.ylabel('Y')
18 plt.grid(True, linestyle='--', alpha=0.6)
19
20 # Plot X^2 vs Y
21 plt.subplot(1, 2, 2)
22 plt.scatter(X**2, Y, color='green', alpha=0.6, edgecolor='k')
23 plt.title('Scatter plot of $X^2$ vs Y')
24 plt.xlabel('$X^2$')
25 plt.ylabel('Y')
26 plt.grid(True, linestyle='--', alpha=0.6)
27
28 plt.tight_layout()
29 plt.show()
```



Task 1 — Loss Function

Given our model $\hat{y}_i = C_0 + C_1 x_i^2$, The loss function we use is the **Mean Squared Error (MSE)**, which computes the average squared differences (residuals) across all data points:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Substituting $\hat{y}_i$ gives our target equation:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)^2$$

Task 2 — Gradient of Each Coefficient

To compute the gradients of the loss function L with respect to each parameter (C_0 and C_1), we apply the **chain rule**. For any coefficient θ , the partial derivative is:

$$\begin{aligned}\frac{\partial L}{\partial \theta} &= \frac{\partial}{\partial \theta} \left[\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right] \\ &= \frac{1}{n} \sum_{i=1}^n 2(y_i - \hat{y}_i) \cdot \frac{\partial}{\partial \theta} (y_i - \hat{y}_i) \\ &= -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot \frac{\partial \hat{y}_i}{\partial \theta}\end{aligned}$$

— 1. Gradient with respect to C_0 :

First, find $\frac{\partial \hat{y}_i}{\partial C_0}$:

$$\begin{aligned}\hat{y}_i &= C_0 + C_1 x_i^2 \\ \frac{\partial \hat{y}_i}{\partial C_0} &= \frac{\partial}{\partial C_0} (C_0 + C_1 x_i^2) = 1 + 0 = 1\end{aligned}$$

Substituting this back into the chain rule formula:

$$\begin{aligned}\frac{\partial L}{\partial C_0} &= -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot 1 \\ \frac{\partial L}{\partial C_0} &= -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)\end{aligned}$$

— 2. Gradient with respect to C_1 :

First, find $\frac{\partial \hat{y}_i}{\partial C_1}$:

$$\begin{aligned}\hat{y}_i &= C_0 + C_1 x_i^2 \\ \frac{\partial \hat{y}_i}{\partial C_1} &= \frac{\partial}{\partial C_1} (C_0 + C_1 x_i^2) = 0 + x_i^2 = x_i^2\end{aligned}$$

Substituting this back into the chain rule formula:

$$\begin{aligned}\frac{\partial L}{\partial C_1} &= -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot x_i^2 \\ \frac{\partial L}{\partial C_1} &= -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2\end{aligned}$$

Task 3 — Update Rules

In each iteration t , the parameters are updated in the opposite direction of the gradient to minimize the loss, scaled by the learning rate η :

$$C_0^{(t+1)} = C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0}$$

$$C_1^{(t+1)} = C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1}$$

Task 4 — Gradient Descent Implementation

In this section, we implement the code to find C_0 and C_1 using basic NumPy, based on the mathematical equations derived in the previous tasks.

In [3]:

```
1 # Hyperparameters
2 learning_rate = 0.01
3 n_iterations = 5000
4 n = len(Y)
5
6 # Initialize parameters
7 C0 = 0.0
8 C1 = 0.0
9
10 # Prepare lists to store history (for visualization later)
11 history_loss = []
12 history_C0 = []
13 history_C1 = []
14
15 # Pre-compute X^2 to reduce computation inside the loop
16 X_sq = X**2
17
18 # Gradient Descent Loop
19 for i in range(n_iterations):
20     # 1. Calculate Predictions
21     Y_pred = C0 + C1 * X_sq
22
23     # 2. Calculate Errors/Residuals
24     error = Y - Y_pred
25
26     # 3. Calculate Loss (MSE)
27     loss = np.mean(error**2)
```

```

28     history_loss.append(loss)
29     history_C0.append(C0)
30     history_C1.append(C1)
31
32     # 4. Calculate Gradients
33     grad_C0 = -(2/n) * np.sum(error)
34     grad_C1 = -(2/n) * np.sum(error * X_sq)
35
36     # 5. Update Parameters
37     C0 = C0 - learning_rate * grad_C0
38     C1 = C1 - learning_rate * grad_C1
39
40     # Print progress every 500 iterations
41     if i % 500 == 0 or i == n_iterations - 1:
42         print(f"Iteration {i:4d} | Loss: {loss:.5f} | C0: {C0:.5f}
43             | C1: {C1:.5f}")
44
45     print("-" * 50)
46     print(f"Training complete over {n_iterations} iterations")
47     print(f"Optimized Parameters: C0 = {C0:.4f}, C1 = {C1:.4f}")
48     print(f"Final MSE Loss: {history_loss[-1]:.6f}")

```

Out[3]:

```

Iteration    0 | Loss: 1.30516 | C0: 0.02239 | C1: 0.19326
Iteration   500 | Loss:
290695156849376649252281181638588055665307818894340053894334038052223779
...
Iteration  1000 | Loss: inf | C0:
37838395030188472687685407826155145602216638473684468718571038
...
Iteration  1500 | Loss: nan | C0: nan | C1: nan
Iteration  2000 | Loss: nan | C0: nan | C1: nan
Iteration  2500 | Loss: nan | C0: nan | C1: nan
Iteration  3000 | Loss: nan | C0: nan | C1: nan
Iteration  3500 | Loss: nan | C0: nan | C1: nan
Iteration  4000 | Loss: nan | C0: nan | C1: nan
Iteration  4500 | Loss: nan | C0: nan | C1: nan
Iteration  4999 | Loss: nan | C0: nan | C1: nan

-----

Training complete over 5000 iterations
Optimized Parameters: C0 = nan, C1 = nan
Final MSE Loss: nan

```

```

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\numpy
_core\methods.py:132: RuntimeWarning: overflow encountered in
ret = umr_sum(arr, axis, dtype, out, keepdims, where=where)
C:\Users\Admin\AppData\Local\Temp\ipykernel_2560\3501735774.py:27:
RuntimeWarning: overflow encountered in
loss = np.mean(error**2)
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\numpy
_core\fromnumeric.py:83: RuntimeWarning: invalid value encountered in
return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
C:\Users\Admin\AppData\Local\Temp\ipykernel_2560\3501735774.py:38:
RuntimeWarning: invalid value encountered in
C1 = C1 - learning_rate * grad_C1

```

Task 5 — Results & Visualizations

In this section, we analyze the behavior of our trained model and evaluate how well it fits the dataset using various plots.

In [4]:

```

1  # Create subplots for a 2x2 grid visualization
2  fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15,
3  12))
4
5  # Plot 1: Learning Curve (Loss over Iterations)
6  ax1.plot(range(n_iterations), history_loss, 'b-', linewidth=2)
7  ax1.set_xlabel('Iteration')
8  ax1.set_ylabel('MSE Loss')
9  ax1.set_title('Loss Function Over Iterations')
10 ax1.grid(True, linestyle='--', alpha=0.6)
11 ax1.set_yscale('log') # Log scale to see convergence better
12
13 # Plot 2: Convergence of parameters C0 and C1
14 ax2.plot(range(n_iterations), history_C0, 'r-', label='C0 (
15 Intercept)', linewidth=2)
16 ax2.plot(range(n_iterations), history_C1, 'g-', label='C1 (
17 Coefficient of X^2)', linewidth=2)
18 ax2.set_xlabel('Iteration')
19 ax2.set_ylabel('Parameter Value')
20 ax2.set_title('Parameter Evolution During Training')
21 ax2.legend()
22 ax2.grid(True, linestyle='--', alpha=0.6)

```

```

21 # Plot 3: Fitted Curve vs Actual Data
22 x_line = np.linspace(min(X), max(X), 100)
23 y_line = C0 + C1 * (x_line**2)
24 ax3.scatter(X, Y, color='blue', alpha=0.6, label='Actual Data',
25             edgecolor='k')
26 ax3.plot(x_line, y_line, 'r-', linewidth=3, label=f'Fitted Model:
27           $\hat{y} = {C0:.4f} + {C1:.4f}x^2$')
28 ax3.set_xlabel('X')
29 ax3.set_ylabel('Y')
30 ax3.set_title('Original Data vs Fitted Model')
31 ax3.legend(fontsize=12)
32 ax3.grid(True, linestyle='--', alpha=0.6)
33
34 # Plot 4: Residuals
35 Y_pred_final = C0 + C1 * X_sq
36 residuals = Y - Y_pred_final
37 ax4.scatter(X, residuals, color='purple', alpha=0.6, edgecolor='k'
38            )
39 ax4.axhline(0, color='red', linestyle='--', linewidth=2)
40 ax4.set_xlabel('X')
41 ax4.set_ylabel('Residuals ($y_i - \hat{y}_i$)')
42 ax4.set_title('Residual Plot')
43 ax4.grid(True, linestyle='--', alpha=0.6)
44
45 plt.tight_layout()
46 plt.show()

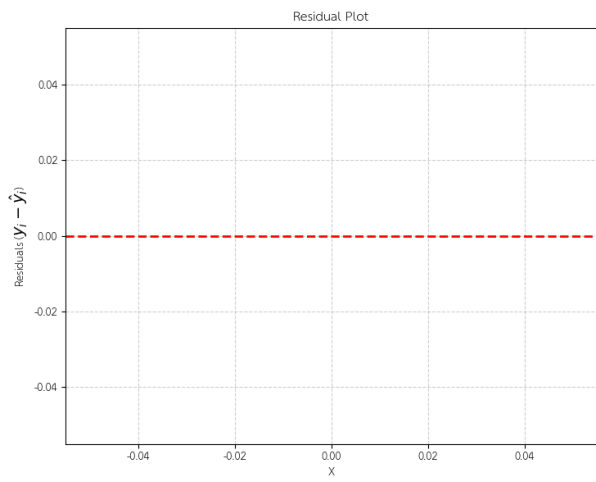
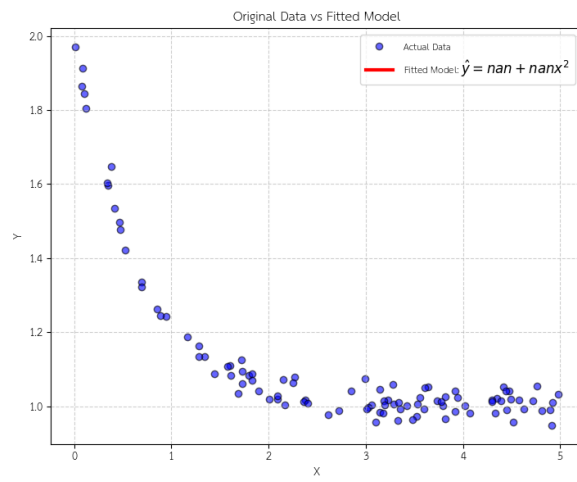
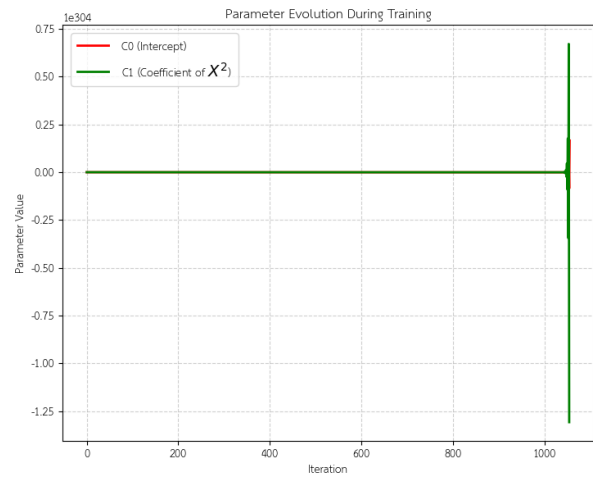
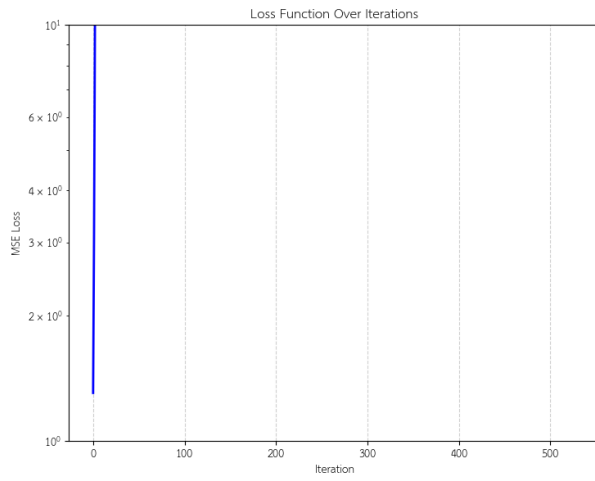
```

Out[4]:

```

<>:25: SyntaxWarning: invalid escape sequence '\h'
<>:38: SyntaxWarning: invalid escape sequence '\h'
<>:25: SyntaxWarning: invalid escape sequence '\h'
<>:38: SyntaxWarning: invalid escape sequence '\h'
C:\Users\Admin\AppData\Local\Temp\ipykernel_2560\1276841915.py:25:
  SyntaxWarning: invalid escap ...
  ax3.plot(x_line, y_line, 'r-', linewidth=3, label=f'Fitted Model:
    $\hat{y} = {C0:.4f} + {C1} ...
C:\Users\Admin\AppData\Local\Temp\ipykernel_2560\1276841915.py:38:
  SyntaxWarning: invalid escap ...
  ax4.set_ylabel('Residuals ($y_i - \hat{y}_i$)')
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\
  matplotlib\scale.py:270: RuntimeW ...
  return np.power(self.base, values)

```



Task 6 — Model Evaluation (R-squared & Summary Statistics)

To mathematically evaluate the goodness of fit for our machine learning model, we calculate the Coefficient of Determination, also known as R^2 . The formula for R^2 is:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

- **Residual Sum of Squares (SS_{res})** is the sum of the squared differences between the actual and predicted values:

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Total Sum of Squares (SS_{tot})** is the sum of the squared differences between the actual values and the mean of the actual values ($\bar{y}$):

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

A higher R^2 score (closer to 1) indicates that our model's predictions closely match the actual data points.

In [5]:

```
1 # Print summary statistics
2 print("\n=== MODEL EVALUATION SUMMARY ===")
3 print(f"Fitted parameters: C0 = {C0:.6f}, C1 = {C1:.6f}")
4 print(f"Total Iterations: {len(history_loss)}")
5 print(f"Initial loss: {history_loss[0]:.6f}")
6 print(f"Final loss: {history_loss[-1]:.6f}")
7 loss_reduction = (history_loss[0] - history_loss[-1]) /
8     history_loss[0] * 100
9 print(f"Loss reduction: {loss_reduction:.2f}%")
10
11 # Calculate R-squared
12 Y_mean = np.mean(Y)
13 ss_res = np.sum((Y - Y_pred_final)**2)
14 ss_tot = np.sum((Y - Y_mean)**2)
15 r_squared = 1 - (ss_res / ss_tot)
16 print(f"R-squared (R^2): {r_squared:.6f}")
```

Out[5]:

```
=== MODEL EVALUATION SUMMARY ===
Fitted parameters: C0 = nan, C1 = nan
Total Iterations: 5000
```

```
Initial loss: 1.305162  
Final loss: nan  
Loss reduction: nan%  
R-squared ( $R^2$ ): nan
```